Benchmark model state equation

- In this lesson, you will learn how to implement the IEKF in Octave, and will see results of applying it to a specific challenging benchmark model.[1]
- The model has state equation:

$$x_{k+1} = 0.5x_k + 25\frac{x_k}{1+x_k^2} + 8\cos(1.2k) + w_k$$

- The derivative that the EKF/IEKF needs from this equation can be found to be:

$$\frac{\partial f(x_k, u_k, w_k)}{\partial x_k} = 0.5 - \frac{50x_k^2}{(1+x_k^2)^2} + \frac{25}{1+x_k^2} = \frac{25.5 - 24x_k^2 + 0.5x_k^4}{(1+x_k^2)^2}.$$

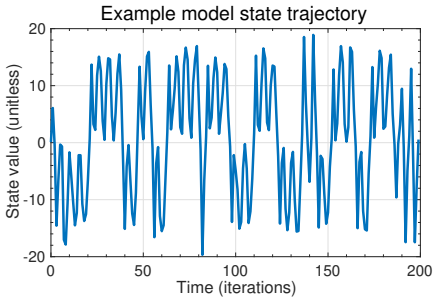
[1] This benchmark is from: Jindřich Havlík and Ondřej Straka. "Performance evaluation of iterated extended Kalman filter with variable step-length." *Journal of Physics: Conference Series*. 659(1), 2015.

Benchmark model output equation

- The benchmark model has output equation: $z_k = \frac{x_k^2}{20} + v_k$.
- The derivative that the EKF/IEKF needs from this equation can be found to be:

$$\frac{\partial h(x_k, u_k, v_k)}{\partial x_k} = \frac{x_k}{10}.$$

- In the examples, $\Sigma_{\tilde{w}} = 1$ and $\Sigma_{\tilde{v}} = 0.01$.
- The true initial state is $x_0 = 0.1$; the EKF/IEKF are initialized with $\hat{x}_0^+ = 0.1$ and $\Sigma_{\tilde{x},0}^+ = 1$.
- An example model state trajectory is shown.



- Note that the (noise-free) output is always positive even though the state can be either positive or negative. This makes estimating the state's sign very difficult.
- EKF/IEKF relies on the state-equation's "momentum" in step 1a and accurate step 2b.

Initialization

- The IEKF initialization code is identical to its EKF counterpart, except that we now include a parameter `NI`, which is the number of iterations to perform each measurement update.

```
function ekfData = initIEKF(xhat0,SigmaX0,priorU,SigmaW,SigmaV,NI)
  ekfData.xhat   = xhat0;    % Initial state description
  ekfData.SigmaX = SigmaX0;  % Initial estimation-error covariance
  ekfData.SigmaV = SigmaV;   % Sensor-noise covariance
  ekfData.SigmaW = SigmaW;   % Process-noise covariance
  ekfData.priorU = priorU;   % Previous value of input force
  ekfData.Qbump  = 5;        % In case SigmaX collapses
  ekfData.NI     = NI;       % Number of iterations of IEKF meas update
end
```

Changes to the state equation

- In the `iterIEKF` function, there are a few differences from the version you have seen before.
- Since the model has changed, we must change the computation of `Ahat` and $\hat{x}_k^-$:

```
% EKF Step 0: Compute Ahat[k-1], Bhat[k-1]
Ahat = (25.5 - 24*xhat^2+0.5*xhat^4)/(1+xhat^2)^2;
Bhat = 1;

% Step 1a: State estimate time update
xhat = 0.5*xhat+25*xhat/(1+xhat^2)+priorU;
```

- Step 1b of the `iterIEKF` function remains unchanged from before.

Iteration code

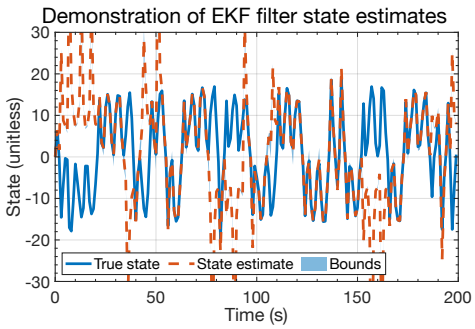
- Then, later in the `iterIEKF` function, we delete the old code for Steps 1c to 2c and replace them with the following:

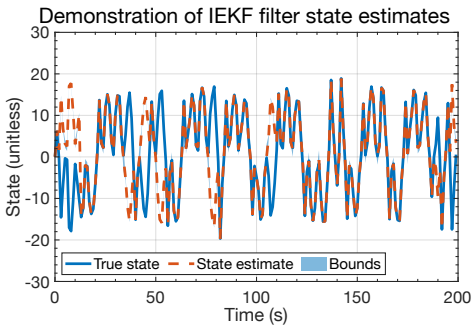
```
% IEKF steps 1c to 2c... iterated
xhatI = xhat;      % Initialize xhat for iteration 0
SigmaXI = SigmaX; % Initialize SigmaX for iteration 0

for j = 0:NI       % The iteration loop
  Chat = xhatI/10; Dhat = 1;                          % Compute derivatives
  zkhat = xhatI^2/20 + Chat*(xhat-xhatI);             % Step 1c
  SigmaZ = Chat*SigmaX*Chat' + Dhat*SigmaV*Dhat';     % Step 2a
  L = SigmaX*Chat'/SigmaZ;                            % Step 2a (continued)
  xhatI = xhat + L*(zk - zkhat);                      % Step 2b
  SigmaXI = (eye(length(xhat))-L*Chat)*SigmaX;        % Step 2c
end

xhat = xhatI;      % The final output of the iteration: xhat
SigmaX = SigmaXI; % and SigmaX
```

- The rest of the `iterIEKF` function remains unchanged from before.

Example EKF/IEKF results

- Some sample results are shown below, comparing EKF and IEKF performance on the same dataset ($N = 5$).
  - Neither is perfect, but the EKF gets confused easier when the sign of x_k changes.
  - EKF had higher RMSE and frequency of the confidence bounds being too tight.





Summary

- You have learned how to implement the IEKF in Octave code.
- Application to a very challenging example has shown that the IEKF can produce meaningful improvement over the EKF.
- The level of improvement will be highly problem-dependent, and is expected to be greatest when a model's output equation is very nonlinear.
- If you are implementing an EKF anyway, it is probably worth trying.